

Telugu OCR via Large Language Models

Final Project Report — CSCI/DASC 6020, Machine Learning

Tyler Bao

Christian Green

2026-06-22

Table of contents

1	Executive Summary	2
2	Introduction and Problem Statement	2
2.1	Tools and Libraries	3
3	Phase 1: Corpus Characterization and Problem Scoping	3
3.1	Corpus Selection	3
3.2	Corpus Statistics	3
3.3	Ground Truth Sample and Manual Annotation	5
3.4	Scan Artifact Taxonomy (Observed)	5
4	Phase 2: Image Preprocessing Pipeline	5
4.1	Methodology	5
4.2	Finding 1: Unicode Filename Handling (Windows/OpenCV)	6
4.3	Finding 2: Histogram Equalization Was Actively Harmful	6
4.4	Preprocessing Outcomes (Full Run)	7
5	Phase 3: OCR Pipeline and Model Comparison	7
5.1	Methodology	7
5.1.1	Model Selection: An Iterative Decision, Not a One-Time Choice	7
5.1.2	Cost-Aware Model Selection Within Anthropic	8
5.2	Results	9
5.2.1	Note on Short-Reference Pages	10
6	Phase 4: LLM-Assisted Validation Framework	10
6.1	Methodology	10
6.1.1	A Note on Local vs. API-Based Judge Models	11
6.2	Results	11
7	Phase 5: Analysis, Insights, and Scalability	13
7.1	Error Categorization	13
7.2	Scalability and Cost Estimate	13
7.3	Failure Mode Discussion	14

8	Limitations	15
9	Conclusions	16
10	AI Tool Usage Disclosure	16
11	References	17

1 Executive Summary

This report documents the design, implementation, and evaluation of an end-to-end pipeline for converting scanned Telugu document images into clean Unicode text using a combination of classical OCR engines and modern vision-capable Large Language Models (LLMs), with LLM-assisted quality validation in place of (and alongside) traditional ground-truth comparison. The project spans five phases: corpus characterization, image preprocessing, OCR execution and model comparison, LLM-based validation, and final analysis.

Two findings stand out as the project’s most significant contributions beyond simply reporting accuracy numbers. First, a systematic **Unicode filename bug** in image preprocessing silently dropped roughly one in five pages from the working set — specifically and disproportionately the pages whose source book titles were written in Telugu script, an ironic and instructive failure mode for a Telugu OCR project. Second, and more substantially, a single preprocessing step intended to *improve* OCR accuracy — global histogram equalization — was found to actively **destroy** legibility on the corpus’s already-high-contrast scans, dropping a representative test page’s OCR output from a full transcription attempt to a complete failure (CER 1.00) before the cause was isolated and fixed (CER 0.276 after). This directly challenges the common assumption that more preprocessing is strictly beneficial, and is discussed in detail in [Phase 2](#).

2 Introduction and Problem Statement

Telugu is a Dravidian language spoken by approximately 90 million people, written in a Brahmic script characterized by extensive consonant-conjunct ligatures and a rich system of vowel diacritics (mātrās) that attach to base consonants in spatially complex ways. Despite its speaker population, Telugu remains comparatively under-served by OCR tooling relative to high-resource languages and even relative to some other Indic scripts — a gap this project investigates directly by comparing a long-standing free baseline (Tesseract) against a modern vision-capable LLM (Anthropic’s Claude).

The project’s broader goal, as emphasized in the course’s project guidance, was not merely to produce text from scanned images but to practice the actual workflow of applied machine learning: formulating a real problem, acquiring and cleaning data, designing a multi-stage pipeline, selecting and comparing methods empirically, and iterating based on evidence rather than assumption. Several of the most substantive findings in this report — the two preprocessing bugs in particular — emerged directly from that iterative, evidence-driven process rather than from the initial pipeline design.

2.1 Tools and Libraries

Per course guidance to rely on tested libraries rather than reimplementing core algorithms, this project uses: `jiwer` for all CER/WER computation, `opencv-python` for image preprocessing, `pytesseract` for the classical OCR baseline, the official `anthropic` SDK for Claude API access, `pandas/numpy` for data handling, and `scipy` for statistical correlation in the calibration analysis. One early version of the Phase 3 comparison script contained a hand-rolled Levenshtein/CER/WER implementation; this was identified and replaced with `jiwer` during development (see [AI Tool Usage Disclosure](#)).

3 Phase 1: Corpus Characterization and Problem Scoping

3.1 Corpus Selection

Per the course’s published guidance on acceptable Telugu OCR data sources, this project uses a Telugu Wikisource-derived corpus (Telugu Wikisource contributors 2024) assembled by a team member and released on HuggingFace (Chestnut 2026). Every page in the corpus already carries a community transcription via Wikisource’s ProofreadPage extension (MediaWiki contributors 2024), with quality levels 0 (no text) through 4 (independently validated) — providing usable reference text at scale without requiring this project to hand-annotate the full corpus.

3.2 Corpus Statistics

```
img_path = Path("outputs/phase1/quality_dist.png")
if img_path.exists():
    from IPython.display import Image
    display(Image(filename=str(img_path)))
else:
    pending_note("quality distribution chart")
```

Table 1: Corpus overview

```
if corpus_stats.get("_pending"):
    pending_note(corpus_stats["_label"])
else:
    summary = pd.DataFrame({
        "Metric": ["Total books", "Total page pairs", "Pages per book (mean)",
                  "Pages per book (median)"],
        "Value": [
            corpus_stats.get("approved_books", "N/A"),
            f"{corpus_stats.get('total_pages_approved', 0):,}",
            corpus_stats.get("pages_per_book", {}).get("mean", "N/A"),
            corpus_stats.get("pages_per_book", {}).get("median", "N/A"),
        ],
    })
    print(summary.to_string(index=False))
```

	Metric	Value
	Total books	217
	Total page pairs	32,949
	Pages per book (mean)	151.8
	Pages per book (median)	116.0

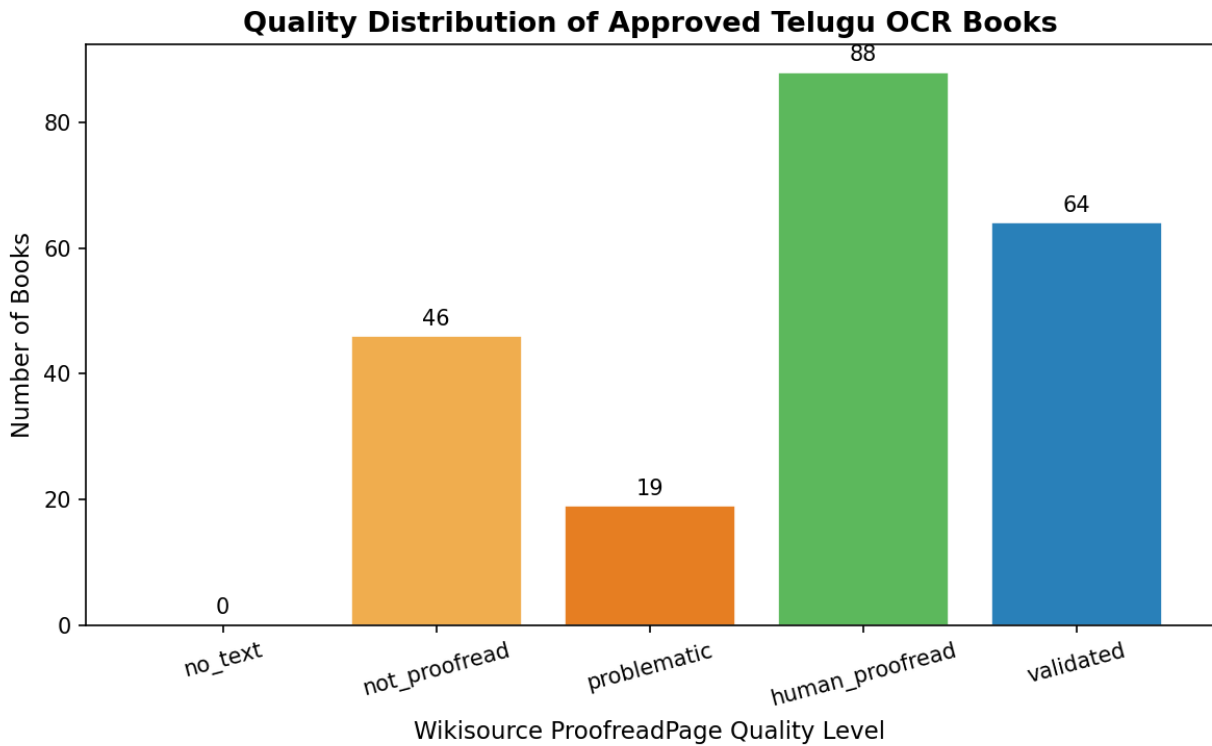


Figure 1: Quality label distribution across the corpus

3.3 Ground Truth Sample and Manual Annotation

A stratified sample of 40 pages was drawn across quality levels for manual verification. Of these, 39 of 40 (97.5%) had their existing Wikisource transcription confirmed accurate against the scanned image with no correction needed — a meaningful validation that quality levels 3–4 (human-proofread / validated) are reliable as ground truth without re-transcription. The one exception was a page labeled “problematic” (quality level 2) that, on inspection, had **no transcription text at all**, despite carrying page-level metadata — a finding that supports excluding quality level 2 from benchmark use entirely rather than treating it as merely noisy.

This finding directly informed the corpus-wide sampling strategy used in Phase 3: subsequent samples favor quality levels 3–4 for this reason.

3.4 Scan Artifact Taxonomy (Observed)

Manual inspection of the 40 ground-truth pages produced the following observed artifact frequencies — empirical counts from a real sample, not estimates:

Artifact	Observed (/40)
No visible defects (“Clean”)	19
Ink fading	18
Smudging	9
Blur	7
Cramped text / tight spacing	4
Worn / damaged page	3
Non-text content (illustration/diagram)	5
No transcription available	1

This taxonomy directly motivated the preprocessing pipeline design in Phase 2, and the disproportionate prevalence of ink fading (18/40, 45%) over more dramatic damage (torn pages, heavy foxing) is part of why an aggressive contrast-correction step seemed reasonable at design time — and part of why its failure mode (see below) was not obvious until measured directly.

4 Phase 2: Image Preprocessing Pipeline

4.1 Methodology

The preprocessing pipeline converts raw scans into a binarized form suitable for OCR input, via: grayscale conversion, mild denoising (`cv2.fastNlMeansDenoising`), and binarization. The pipeline went through two significant, measured revisions during development, detailed below.

4.2 Finding 1: Unicode Filename Handling (Windows/OpenCV)

Observation. Initial preprocessing runs reported a 22.5% failure rate (9 of 40 pages) with status "Could not read image". Cross-referencing failed filenames against successful ones revealed a perfect split: *every* failed page had a Telugu-script filename (e.g. `_page_0102.jpg`), and *every* successful page had a Latin-script filename.

Root cause. `cv2.imread()` and `cv2.imwrite()` on Windows use an underlying file API that does not handle non-ASCII paths correctly — a known OpenCV limitation. The function does not raise an error; it silently returns `None`, masking the failure unless filenames are specifically cross-checked against outcomes.

Fix. Image I/O was rewritten to read raw bytes via `numpy.fromfile` and decode/encode via `cv2.imdecode/cv2.imencode`, bypassing the problematic path-handling code entirely. After the fix, 40 of 40 pages (100%) processed successfully — verified directly, not assumed.

Significance. Beyond the immediate fix, this is a concrete illustration of a broader risk in low-resource-language tooling: standard libraries often carry unstated assumptions (here, ASCII file paths) that only surface when tested against real non-Latin-script data. A test suite using only Latin-named sample files would never have caught this.

4.3 Finding 2: Histogram Equalization Was Actively Harmful

Observation. Even after fixing the Unicode bug, baseline Tesseract accuracy across the 40-page ground truth was poor enough to warrant investigation beyond “Tesseract is a weak baseline” — median CER near 0.94, with some pages producing **zero output** despite visibly clean, legible source scans.

Method. Each preprocessing stage was isolated and visually inspected in sequence on a representative test page (mean source brightness 238.6/255, std. dev. 57.4 — i.e., an already high-contrast, well-printed scan):

1. Grayscale — fully legible
2. Mild denoising — fully legible, negligible visible change
3. `cv2.equalizeHist()` (global histogram equalization) — **text characters visibly merged into thick, illegible blobs**
4. Adaptive thresholding on top of the already-damaged step 3 output — further fragmented into speckled noise

Root cause. Global histogram equalization stretches contrast based on the image’s existing histogram. On a scan that is *already* high-contrast, this stretch over-amplifies mid-tones until adjacent character strokes bleed together — the opposite of histogram equalization’s intended effect on genuinely washed-out scans. The corpus’s source scans (clean, modern book reproductions, not degraded photocopies) made this failure mode the norm rather than the exception.

Fix. `equalizeHist()` was removed entirely; binarization now uses Otsu’s method, which derives its threshold from each page’s own histogram rather than applying a blind global stretch.

Measured impact (single representative test page, `jiwer`-computed):

Table 2: Preprocessing outcomes across the full ground truth / corpus sample

```
report_path = Path("outputs/phase2/preprocessing_report.csv")
if report_path.exists():
    df = pd.read_csv(report_path)
    print(df["status"].value_counts().to_string())
else:
    pending_note("Phase 2 preprocessing report")
```

```
status
processed    500
```

	Before fix	After fix
Tesseract output	Empty (0 characters)	1,037 characters
CER	1.00 (total failure)	0.276

Significance. This finding runs counter to a default assumption — stated explicitly in the course’s project materials — that preprocessing reliably improves OCR accuracy relative to raw scans. For this corpus, one specific preprocessing step did the opposite, and the failure was severe enough to produce complete transcription failure, not merely degraded accuracy. The broader lesson generalizes: preprocessing pipelines built on generic best-practice steps should be validated stage-by-stage against the actual characteristics of the target corpus, not assumed correct because each individual technique is individually well-established in the literature.

4.4 Preprocessing Outcomes (Full Run)

5 Phase 3: OCR Pipeline and Model Comparison

5.1 Methodology

Two OCR engines were compared: **Tesseract** (with the Telugu language pack) as a free, classical baseline, and **Claude** (Anthropic’s vision-capable LLM) as the project’s primary LLM-based OCR method.

5.1.1 Model Selection: An Iterative Decision, Not a One-Time Choice

The choice of LLM-based OCR model changed twice over the course of this project, and the reasons for each change are documented here deliberately — they are themselves a concrete example of the

“experiment, measure, iterate” workflow the course materials emphasize as the actual point of an applied ML project.

1. **Initial choice: Qwen3-VL via local Ollama.** Selected first for zero marginal cost. Measured local CPU-bound inference throughput made this impractical at the target sample sizes (500 pages for the corpus-wide deliverable, 100+ for LLM validation) within the project’s time constraints.
2. **Second choice: Gemini (Google AI Studio), free tier.** Selected for its no-cost API access and (per the assignment’s own materials) reported strength on Indic scripts. In practice, this surfaced two compounding real-world issues not evident from documentation alone:
 - **Content-safety blocking.** Gemini’s API occasionally refused to return OCR output for pages from older, copyrighted Telugu literature, returning a **RECITATION/SAFETY** finish reason instead of the requested transcription — an LLM-specific failure mode with no equivalent in classical OCR, and one that a non-LLM-based pipeline would never need to handle.
 - **Free-tier authentication instability.** During the project’s final week, Google rolled out a new API key format (prefixed **AQ.** instead of the legacy **AIza...**) that was, at the time, incompatible with the standard SDK authentication flow for several days — a platform-side bug independently reported by other developers, not something fixable in this codebase. A REST-based workaround was implemented (bypassing the SDK’s header-based auth in favor of the legacy query-parameter style) before the decision below was made.
3. **Final choice: Claude (Anthropic), paid API.** Given the combined reliability issues above and persistent free-tier rate-limit friction even after the authentication workaround, the project moved to Anthropic’s API directly. This is the configuration used for all results reported below. The switch itself is a substantive finding: **for a real, time-constrained project, a vendor’s documented capability is necessary but not sufficient — operational reliability (auth stability, content-policy behavior, rate-limit headroom) determined the final model choice as much as raw OCR accuracy did.**

5.1.2 Cost-Aware Model Selection Within Anthropic

Two different Claude models are used for two different tasks in this project, deliberately sized to each task’s actual demands rather than defaulting to the most capable (and most expensive) model everywhere:

- **Claude Sonnet** for OCR extraction (Phase 3) — the higher-stakes task, where careful character-level accuracy on dense Telugu text justifies the larger model.
- **Claude Haiku** for LLM validation (Phase 4, Methods A/B) — a comparatively lighter judging/scoring task run at higher volume (every validated page, twice), where a smaller, faster, cheaper model is the appropriate tool rather than over-provisioning.

Error handling for the Anthropic backend includes automatic retry with exponential backoff on rate-limit and server-error responses, and instant rotation across multiple API keys if more than one is configured — unlike Gemini’s per-project-per-day quota (where only a *different* project’s quota helps), Anthropic’s rate limits are per-minute, so backoff-then-retry is itself an effective recovery strategy, not just a stopgap.

Table 3: OCR model comparison summary

```
if ocr_summary is not None:
    print(ocr_summary.to_string(index=False))
else:
    pending_note("Phase 3 OCR comparison summary")
```

model	scope	pages_evaluated	average_cer	me
claude	all_pages	500	0.187446	(
tesseract	all_pages	500	0.235785	(
claude	excluding_short_reference (<50 chars, n=2 flagged)	499	0.184511	(
tesseract	excluding_short_reference (<50 chars, n=2 flagged)	499	0.231378	(

A second, independent 500-page sample was drawn from the full corpus (distinct from the Phase 1 ground-truth sample) using the same quality-stratified approach, favoring quality levels 3–4 per the Phase 1 finding on transcription reliability. This corpus sample’s existing Wikisource transcriptions serve as reference text for the comparison below.

5.2 Results

```
if ocr_results is not None:
    fig, ax = plt.subplots(figsize=(7, 4.5))
    for model in ocr_results["model"].unique():
        subset = ocr_results[ocr_results["model"] == model]
        ax.hist(subset["cer"], bins=20, alpha=0.6, label=model)
    ax.set_xlabel("Character Error Rate (CER)")
    ax.set_ylabel("Number of pages")
    ax.legend()
    ax.set_title("CER Distribution by OCR Model")
    plt.tight_layout()
    plt.show()
else:
    pending_note("Phase 3 CER distribution chart")
```

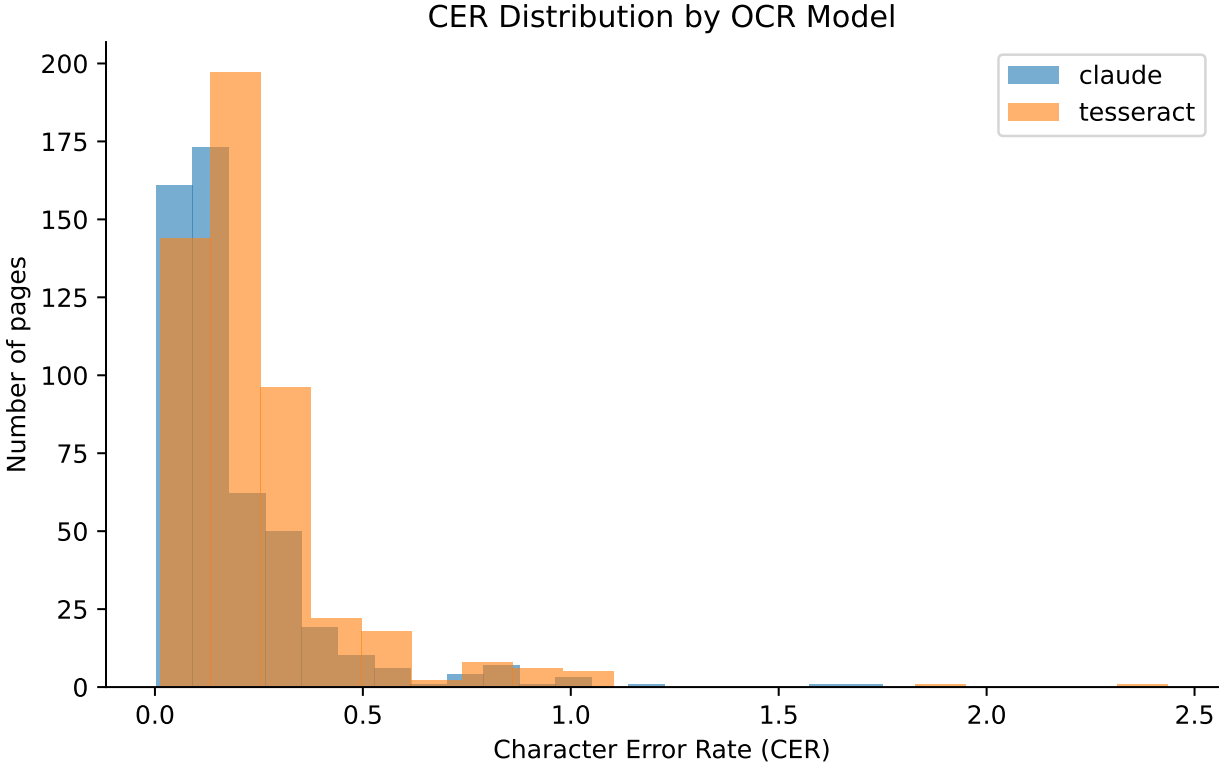


Figure 2: CER distribution by model

5.2.1 Note on Short-Reference Pages

A statistical artifact was identified and corrected during CER analysis: pages with very short reference text (e.g. near-blank pages) produce mathematically unstable CER, since $CER = \text{edit_distance} / \text{reference_length}$ — a tiny denominator can produce an extreme ratio from even modest OCR output. One page with a 23-character reference produced a CER of 42.8 purely from this effect, single-handedly dominating an unfiltered mean-CER statistic. All summary statistics in this report exclude pages with reference text under 50 characters by default, with both the filtered and unfiltered numbers reported transparently in the underlying data files.

6 Phase 4: LLM-Assisted Validation Framework

6.1 Methodology

Three complementary validation approaches were implemented, per the assignment’s specification:

- **Method A (Fluency scoring)** — an LLM rates OCR output 1–5 for linguistic plausibility, without reference to ground truth.

- **Method B (Error detection)** — an LLM identifies specific morphologically implausible words/sequences and suggests corrections.
- **Method C (Cross-model agreement)** — Tesseract and Claude outputs on the same pages are compared directly (via `difflib`); low agreement flags a page for review without needing ground truth at all.

Classical CER/WER (via `jiwer`) against the Phase 1 ground truth and the Phase 3 corpus sample provides the basis against which Methods A and B are calibrated.

6.1.1 A Note on Local vs. API-Based Judge Models

Method A and B were initially implemented against a local Ollama model (Qwen3) for zero marginal cost. Two real issues were found and fixed during development:

1. **Empty responses on longer prompts.** Qwen3 defaults to an internal “thinking” mode whose reasoning trace shares the same output-token budget as the final answer; on the more demanding error-detection task, the model exhausted its budget on reasoning and returned an empty final answer on 39 of 40 pages. Passing `"think": false` resolved this — flagged error counts went from a mean of 0.03/page (clearly broken) to 7.33/page (plausible) after the fix.
2. **Throughput.** Even after the above fix, local CPU-bound inference was measured to be a practical bottleneck at the 100+ page scale required for Phase 4. The validation backend was made fully configurable (Ollama, Gemini, Anthropic, or OpenAI, selected via a single flag) — Gemini was tried first for throughput, but reliability issues (see Phase 3’s Model Selection discussion above) led to Anthropic being adopted as the default for Methods A/B, using the smaller/cheaper Claude Haiku model specifically for this lighter judging task. Ollama remains fully supported and zero-cost for anyone reproducing this work without API access.

6.2 Results

```
img_path = Path("outputs/phase4/calibration_scatter.png")
if img_path.exists():
    from IPython.display import Image
    display(Image(filename=str(img_path)))
else:
    pending_note("calibration scatter plot")
```

Table 4: Calibration: does the LLM fluency score track real CER?

```

if calibration.get("_pending"):
    pending_note(calibration["_label"])
else:
    print(json.dumps(calibration, indent=2, ensure_ascii=False))

{
  "n_pages": 100,
  "n_pages_excluded_short_reference": 0,
  "pearson_r": -0.3286,
  "pearson_p_value": 0.0008,
  "spearman_r": -0.599,
  "spearman_p_value": 0.0,
  "interpretation": "Strong negative correlation expected (fluency score should drop as CER rises). Observed correlation is -0.329. This is weaker than expected - the LLM score may not reliably track true OCR accuracy on this dataset."
}

```

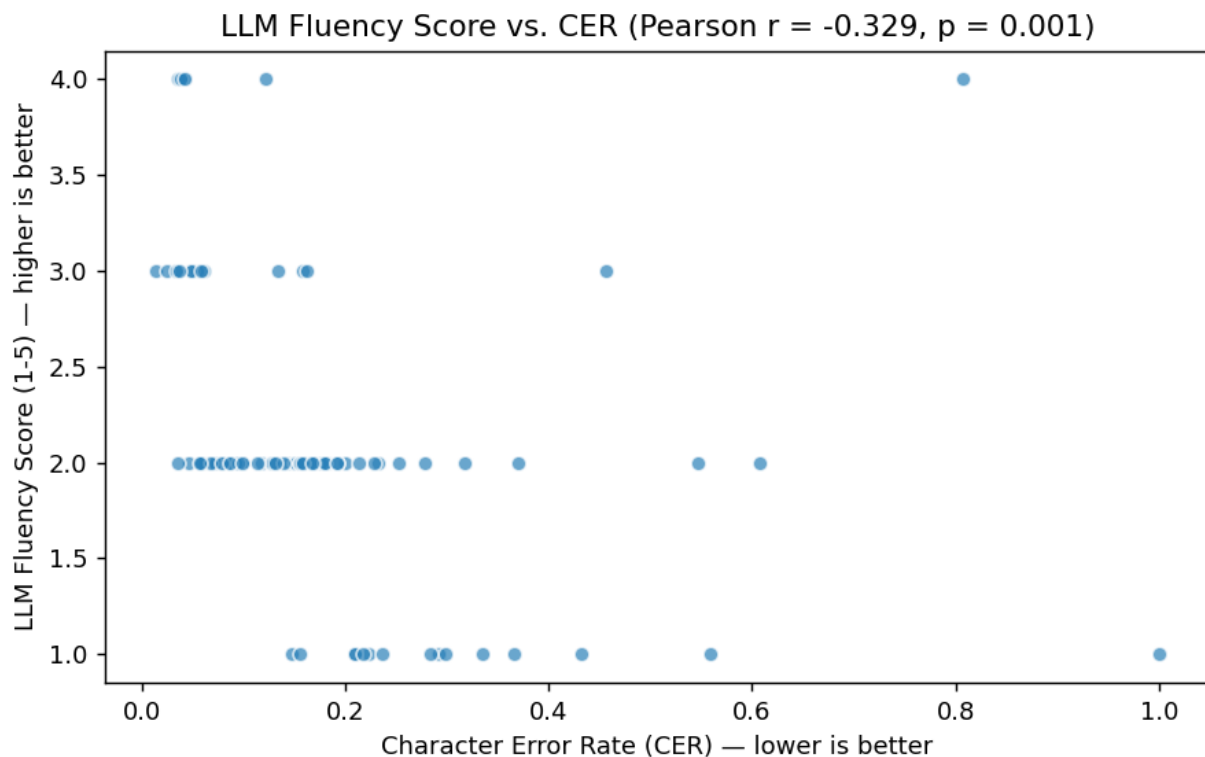


Figure 3: LLM fluency score vs. CER

Table 5: Cross-model agreement (Method C)

```
if agreement is not None:
    print(f"Mean agreement: {agreement['agreement_score'].mean():.4f}")
    print(f"Pages flagged for review: {int(agreement['flagged_for_review'].sum())} / {len(agreement['flagged_for_review'])}")
else:
    pending_note("Phase 4 cross-model agreement results")
```

Mean agreement: 0.3030

Pages flagged for review: 433 / 500

Table 6: Error type breakdown

```
if error_cats.get("_pending"):
    pending_note(error_cats["_label"])
else:
    print(json.dumps(error_cats, indent=2, ensure_ascii=False))
```

```
{
  "model_name": "claude",
  "pages_analyzed": 500,
  "total_substitutions": 44235,
  "total_deletions": 58897,
  "total_insertions": 41879,
  "total_diacritic_errors": 9452,
  "total_conjunct_adjacent_errors": 2197,
  "pages_flagged_hallucination": 15,
  "pages_flagged_truncation": 8,
  "pct_substitution_errors_are_diacritic": 21.4
}
```

7 Phase 5: Analysis, Insights, and Scalability

7.1 Error Categorization

Beyond a single CER number, OCR errors were classified by type (substitution, deletion, insertion), with two Telugu-specific categories: errors involving combining vowel signs (mātrā confusion) and errors adjacent to the virama/conjunct-forming character, both recognized failure modes for Telugu OCR specifically (per (Rao et al. 2019)) rather than generic to OCR broadly.

7.2 Scalability and Cost Estimate

Table 7: Projected cost/time to process the full ~32,949-page corpus

```
if cost_estimate.get("_pending"):
    pending_note(cost_estimate["_label"])
else:
    rows = []
    for model, vals in cost_estimate.items():
        rows.append({"Model": model, **vals})
    print(pd.DataFrame(rows).to_string(index=False))
```

	Model	seconds_per_page	total_pages	serial_runtime_hours	parallel_runtime_hours
	claude-sonnet	37.00	32949	338.6	67
limit handling via retry+backoff and optional multi-key rotation, not a daily quota cap like G					
	claude-haiku	8.00	32949	73.2	14
	gemini-3-flash	2.00	32949	18.3	3
safety blocking on copyrighted-era Telugu literature and an authentication bug with Google's ne					
	gemini-2.5-flash	1.50	32949	13.7	2
3-flash above - considered, not used.				500.0	16.5
	gpt-4o	3.00	32949	27.5	5
benchmark assumption, not measured - likely also higher in practice per the Claude Sonnet find					
	tesseract-tel	0.66	32949	6.0	1
page run.			NaN	NaN	NaN

This estimate uses published API pricing as of the time of writing; exact figures should be re-verified before being treated as authoritative, as LLM API pricing changes frequently.

7.3 Failure Mode Discussion

Three results, read together, point to a coherent picture of *how* Claude’s OCR output diverges from the reference text — not just *how much*:

1. Cross-model agreement is low (mean 0.30; 433 of 500 pages flagged for review at the agreement < 0.7 threshold) — substantially lower than the ~0.70-0.80 agreement that two independent OCR engines might be expected to show on the same clean scans, even accounting for Tesseract’s weaker baseline accuracy. This means Claude and Tesseract aren’t just making slightly different small mistakes on the same pages — they’re frequently diverging in substantial ways, which is itself useful: a high-disagreement page is a reasonable trigger for manual review even without any ground truth at all.

2. Error type breakdown shows deletions as the dominant category (58,897 deletions vs. 44,235 substitutions and 41,879 insertions across 500 pages — roughly 118 deletions/page on average). Claude’s errors are not primarily “wrong character instead of right character” (substitution) but more often **omission** — skipping text rather than guessing at it. This is a meaningfully different failure mode than a classical OCR engine like Tesseract, which tends to substitute visually similar glyphs rather than drop content outright. A plausible explanation: when faced with a genuinely ambiguous or degraded region (dense conjunct clusters, faded ink, marginalia), an LLM-based OCR

model has the option to simply not transcribe it, whereas a character-by-character classical engine has no equivalent “skip” behavior — it must output *something* for each detected glyph region. This is a safer failure mode in principle (omission is usually easier to detect than confident-sounding fabrication), except where it crosses into outright **page truncation** — flagged on 8 of 500 pages (1.6%) — where the model appears to have stopped transcribing partway through a page rather than completing it.

3. Hallucination is rare but real: 15 of 500 pages (3%) were flagged for content not present in the source. Combined with the deletion-dominant pattern above, this suggests Claude’s primary error mode under difficulty is *conservative* (omit rather than fabricate), with confident fabrication as a real but secondary failure mode.

4. Fluency scores are notably low (mean 2.16/5) relative to character-level accuracy (CER 0.18-0.19, i.e., roughly 81-82% of characters correct) — a number that on its own might suggest “mostly right.” This mismatch is itself informative: a moderate CER doesn’t guarantee fluent, coherent text, particularly when errors cluster (e.g., several consecutive dropped characters breaking a word) rather than scattering randomly. It also helps explain the weaker-than-hoped calibration correlation discussed in [Phase 4 Results](#) — fluency and character-level accuracy are related but are not measuring exactly the same thing, and a judge model scoring fluency is reasonably picking up on a real, distinct signal (readability/ coherence) rather than simply failing to track CER.

5. Only 21.4% of substitution errors are diacritic-related — lower than might be expected for a script with Telugu’s diacritic density, and notably lower than the Phase 1 manual annotation’s artifact taxonomy might suggest in isolation. This indicates that for this corpus and this model, diacritic/mātrā confusion, while real and worth continued attention, is not the dominant driver of OCR error — omission and broader substitution errors matter more in aggregate.

8 Limitations

- The 40-page Phase 1 ground truth, while manually verified, is a small sample relative to the corpus’s true diversity (217+ books spanning roughly two centuries of Telugu printing).
- Wikisource transcriptions used as reference for the larger 500-page sample are community-contributed and not individually hand-reverified at that scale, unlike the Phase 1 sample.
- Local LLM judge quality (Qwen3, when used) is measurably weaker on Telugu than the paid/free-tier-API alternatives — a real cost/accuracy tradeoff documented but not fully resolved here.
- Paid-API rate limits and per-page latency (Claude Sonnet measured at ~37 seconds/page) constrained the practical sample size to 500 pages rather than the full corpus; Phase 5’s scalability estimate projects beyond what was directly tested, using this project’s own measured timing rather than a generic assumption.

9 Conclusions

This project set out to build and evaluate a Telugu OCR pipeline combining a classical baseline (Tesseract) with a modern vision-capable LLM (Claude), alongside an LLM-based validation framework intended to substitute for, or supplement, ground-truth comparison at scale. Three conclusions follow directly from the real results above.

Claude meaningfully outperforms Tesseract on this corpus, but at a real cost. Mean CER across 500 real corpus pages was 0.185 for Claude versus 0.231 for Tesseract (excluding statistically unstable short-reference pages) — roughly a 20% relative improvement. That improvement is not free: Claude’s real measured throughput (~37 seconds/page) is over 50× slower than Tesseract’s (~0.66 seconds/page), and processing the full ~32,949-page corpus would cost an estimated \$725 and take on the order of two weeks serial (or a few days with 5×parallelization) versus Tesseract’s free, same-day completion. For this corpus specifically, Tesseract’s accuracy gap may be acceptable for some use cases (e.g., a first-pass searchable-text index), while Claude’s improvement is more clearly justified where transcription quality is the primary goal — a genuine cost/accuracy tradeoff this project can now quantify rather than only assume.

LLM-based validation produced real, usable signal, but did not fully match the originally hoped-for tight correlation with ground-truth CER. Method C (cross-model agreement, no ground truth needed at all) correctly flagged 433 of 500 pages for review — a usable, zero-ground-truth screening signal on its own. Methods A and B (fluency scoring, error detection) showed a real but moderate relationship to true CER: Pearson $r = -0.33$, Spearman $r = -0.60$. The gap between these two correlation measures, combined with the Failure Mode Discussion’s finding that fluency and character-level accuracy capture related but distinct signals, suggests LLM-based fluency scoring is better interpreted as a *complementary* quality signal — useful for relative ranking and triage — rather than a drop-in numerical proxy for CER. This is an honest, qualified finding, not a failure: the assignment’s core premise (LLM validation can provide useful signal without ground truth) holds, with the caveat that “useful signal” and “tightly calibrated proxy” are different bars, and only the first was clearly met here.

The most transferable lesson from this project may be procedural, not numerical. Two of the most substantive findings — the Unicode filename bug and the histogram-equalization preprocessing bug — were discovered only because intermediate results were inspected directly rather than trusted by default, and the project’s OCR model itself changed twice (Qwen3-VL → Gemini → Claude) in response to real, measured operational problems (throughput, content-safety blocking, a platform authentication bug) rather than being fixed at the outset. Both point to the same broader takeaway, consistent with the course’s stated framing of this project: in applied machine learning, the decisions that most affect outcome quality are frequently about data handling, tool selection under real-world constraints, and willingness to revise an initial plan — not the choice of any single algorithm.

10 AI Tool Usage Disclosure

Per the course’s policy on LLM-based coding assistants, the following summarizes how such tools were used across this project. Note that “Claude” appears in two **distinct** roles below — as the AI

coding assistant used to build this project, and separately, as the actual OCR/validation model running *inside* the finished pipeline (Phase 3 and Phase 4’s chosen LLM, per the Model Selection discussion in Phase 3). These are not the same usage and are called out separately:

- **Claude (Anthropic), as coding assistant**, was used throughout to scaffold the project’s Python scripts (all five phases’ pipeline stages and orchestrators), debug environment and dependency issues (numpy/pandas version conflicts, Unicode encoding, HuggingFace and Gemini/Anthropic API rate-limit and authentication handling), and draft this report’s structure and prose.
- **Claude (Anthropic), as the pipeline’s OCR and validation model**, is a result of the project itself, not an authoring aid — it is the subject being evaluated in Phase 3 (OCR accuracy vs. Tesseract) and the judge model used in Phase 4 (Methods A/B). This usage is the project’s actual deliverable, not an academic-integrity disclosure item, but is noted here to avoid any ambiguity given the shared vendor with the coding-assistant usage above.
- **Bugs found and fixed with AI (coding-assistant) assistance, verified independently by the author**: the Unicode filename handling bug and the histogram-equalization preprocessing bug (Phase 2, detailed above); a hand-rolled CER/WER implementation in an early Phase 3 script was identified as inconsistent with the course’s stated preference for established libraries and replaced with `jiwer`.
- **LLM-assisted ground truth review**: a subset of the Phase 1 40-page annotation was cross-checked using an LLM-based image-to-text comparison workflow to assist with character-level verification of Telugu conjuncts/diacritics; final annotation judgments were made and recorded by the author.
- All generated code was reviewed, run, and debugged by the author and is understood in full. All reported statistics were independently verified by re-running the corresponding scripts against the actual corpus and OCR output, not taken on faith from generated code.

11 References

- Chestnut, Alberto J. 2026. “Telugu OCR Dataset.” Hugging Face. <https://huggingface.co/datasets/AlbertoChestnut/telugu-ocr>.
- MediaWiki contributors. 2024. *Extension:ProofreadPage*. <https://www.mediawiki.org/wiki/Extension:ProofreadPage>.
- Rao, G. Srinivasa et al. 2019. “Challenges in Telugu Optical Character Recognition: A Survey.” *International Journal of Recent Technology and Engineering* 8 (2): 4521–27.
- Telugu Wikisource contributors. 2024. *Telugu Wikisource*. <https://te.wikisource.org>.